FUSE AI-201
AI Model Development and Implementation

Group 5

AI Project Implementation Report

Real-Time Network Intrusion Detection Using Multi-Class Machine Learning Classification

Assigned Domain: Cyber security

Group Members:

1. Wogari Ararsa
2. Amanuel Ayalew Arega
3. Eyob Daniel Asefa
4. Surafiel Habib Asefa
5. Mulatie Yeshu Asegie
6. Asamn Admasu Asfaw
7. PETROS TARIKU ASSEFA
8. Semegn Mulugeta Assefa
9. Fuad KedirAssen
10. Mohammed Yassin Awel
11. Tenaw Workie Awoke

Oct 19/20026

Addis Ababa

Table content

1. Problem statement 2

2. Target users 2

3. Proposed AI solution 3

4. Dataset 4

5. Data preparation 5

6. Exploratory data analysis (key findings) 6

7. Baseline model 9

8. Model improvement 9

9. Final model 10

10. Evaluation (held-out KDDTest+, n=22,544) 11

11. Testing (whole system) 15

12. Working prototype 15

13. Limitations and responsible-use notes 16

14. Future improvements 16

15. Conclusion 17

16. References 17

1. Problem statement

Organizations need a practical way to classify network traffic as normal or as a specific attack category in near real time, so analysts can prioritize investigation. Traditional controls signature matching, hand-written rules, threshold anomaly detection, and manual review miss novel/modified attacks, need constant expert maintenance, generate false alarms, and do not scale. A supervised multi-class classifier trained on historical connection records can act as fast, consistent decision support (never an autonomous blocking control). The system classifies one connection record into Normal, Denial-of-Service (DoS), Probe/scanning, Remote-to-Local (R2L), or User-to-Root (U2R), with class probabilities and a plain-language caution.

2. Target users

Stakeholder	**Role**	**Need / value**	**Decision supported**
Security analysts	Investigate alerts	Faster triage of malicious traffic	Which alerts to investigate first
Network administrators	Keep network up	Visibility of disruptive traffic	Review controls / isolate hosts
SOC personnel	Continuous monitoring	Consistent classification	Escalate / prioritize incidents
IT managers	Own technology risk	Summary of intrusion patterns	Resource & planning decisions
Developers / data scientists	Build & evaluate	Structured data + measurable goals	Select & improve model

Primary prototype users: security analysts and network administrators. Output is advisory only.

3. Proposed AI solution

Task: supervised multi-class classification (5 classes). Input: 41 NSL-KDD connection features available at analysis time. Output: predicted category + per-class probabilities + evaluation context. Workflow/architecture:

User → Interface (Streamlit form / CSV upload) → Input validation → Saved preprocessing (One-Hot for protocol_type, service, flag; numeric passthrough) → Trained model (HistGradientBoosting) → Prediction + probabilities + latency → Result display with limitations disclaimer.

Variable group	Feature or target?	Type	Purpose
protocol_type, service, flag	Feature	Categorical (3) — one-hot, unknown ignored	Protocol/service/status signal
duration, src/dst bytes, counts, rates, login/host stats (38)	Feature	Numeric	Volume, error, login and traffic behaviour
category5 (Normal/DoS/Probe/R2L/U2R)	Target	Categorical (5)	What is predicted

Why this approach: tabular data with nonlinear interactions and severe imbalance; tree/ensemble models handle mixed types without scaling, give strong baselines on CPU, and (for RF) feature importance. Logistic Regression / Decision Tree serve as interpretable baselines; Random Forest was the proposal's primary candidate and HistGradientBoosting the gradient-boosting alternative final selection was evidence-driven (see 9). No generative AI is used.

4. Dataset

Primary files used for modelling (full detailed labels): data/KDDTrain+.txt (125,973 records) and data/KDDTest+.txt (22,544 records), NSL-KDD benchmark (Tavallaee et al., 2009), obtained via the public defcom17/NSL_KDD mirror of the UNB/CIC source; Kaggle hosts the same benchmark. Format: comma-separated, no header; 41 features + label + difficulty. Provided course file data/KDDTest_binary.arff (22,588 rows, class ∈ {normal, anomaly}) was inspected and retained, but it collapses all attacks to 'anomaly' so it cannot support the approved 5-class task — hence the full Train+/Test+ pair (same benchmark family) is used and the change is logged here as required by Module 5 Rule #1. License: public research/educational use with citation; no personal data collected.

Label mapping applied (standard NSL-KDD grouping): DoS = back, land, neptune, pod, smurf, teardrop, mailbomb, apache2, processtable, udpstorm, worm; Probe = ipsweep, nmap, portsweep, satan, mscan, saint; R2L = ftp_write, guess_passwd, imap, multihop, phf, spy, warezclient, warezmaster, sendmail, named, snmpgetattack, snmpguess, xlock, xsnoop, httptunnel; U2R = buffer_overflow, loadmodule, perl, rootkit, ps, sqlattack, xterm; else Normal. Test contains 17 attack types absent from train (e.g., mailbomb, mscan, saint, httptunnel, snmp*), making it a genuine novelty test.

Item	Information
Records / variables	Train 125,973 × 43 · Test 22,544 × 43 (41 features + label + difficulty)
Target	category5: Normal / DoS / Probe / R2L / U2R
Missing values	0 in both files (verified)
Duplicates	0 exact duplicates in both files (this NSL-KDD distribution)
Class balance	Train: Normal 53.5%, DoS 36.5%, Probe 9.3%, R2L 0.79%, U2R 0.04% · Test: Normal 43.1%, DoS 33.1%, R2L 12.8%, Probe 10.7%, U2R 0.30%

Known issues	Extreme imbalance (U2R: 52 train rows); correlated rate features; train/test shift (novel attacks); historical data, not modern encrypted/cloud traffic

5. Data preparation

Step	Action	Reason
Missing values	None found; pipeline tolerates None via one-hot unknown handling	Avoid inventing data
Duplicates	Counted before/after (0 removed)	Record honestly
Inconsistent data	Lower-cased labels for mapping; categories passed through OneHotEncoder(handle_unknown='ignore')	Robust to unseen services/flags at serve time
Encoding	One-hot (unordered) for protocol_type/service/flag	Correct for nominal variables
Scaling	No scaling for tree models; StandardScaler only inside the Logistic Regression baseline branch	Trees don't need it; logreg does
Outliers	Kept (e.g., large byte counts are attack signal); verified extreme-value handling in system test	Don't delete signal

Split / leakage	Train = KDDTrain+.txt, held-out test = KDDTest+.txt (benchmark design); 3-fold stratified CV + 2-fold randomized search on train only; preprocessing fit on train folds only	No test leakage; resampling/weights applied to training data only

Reproducibility: fixed random_state=42 everywhere; single saved artifact models/nids_pipeline.joblib bundles preprocessing + estimator so the app applies identical transforms.

6. Exploratory data analysis (key findings)

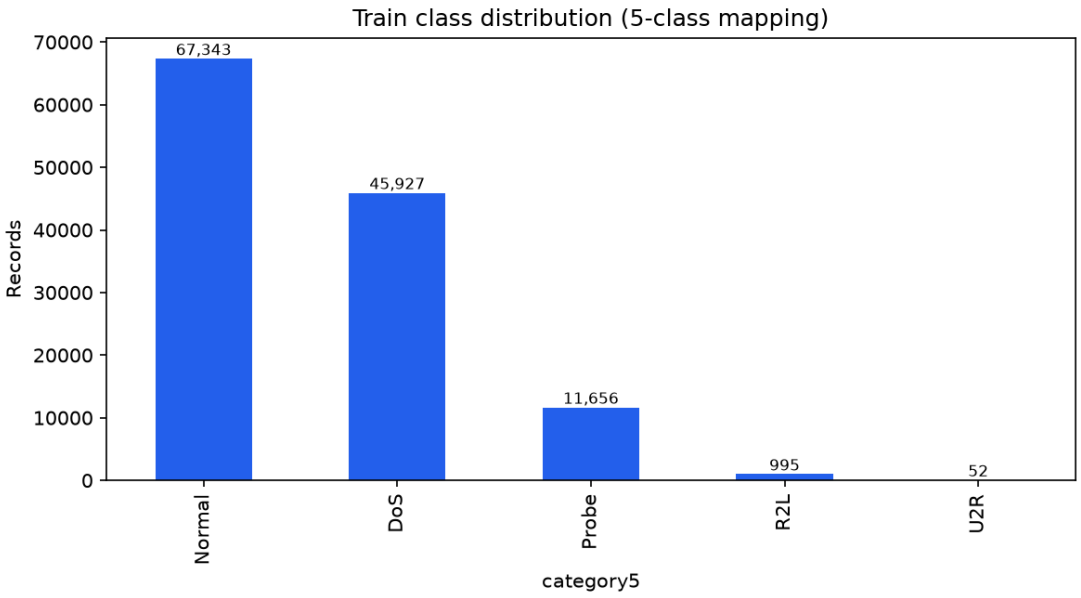


Figure 1 — Train class distribution: Normal and DoS dominate; R2L/U2R are rare.

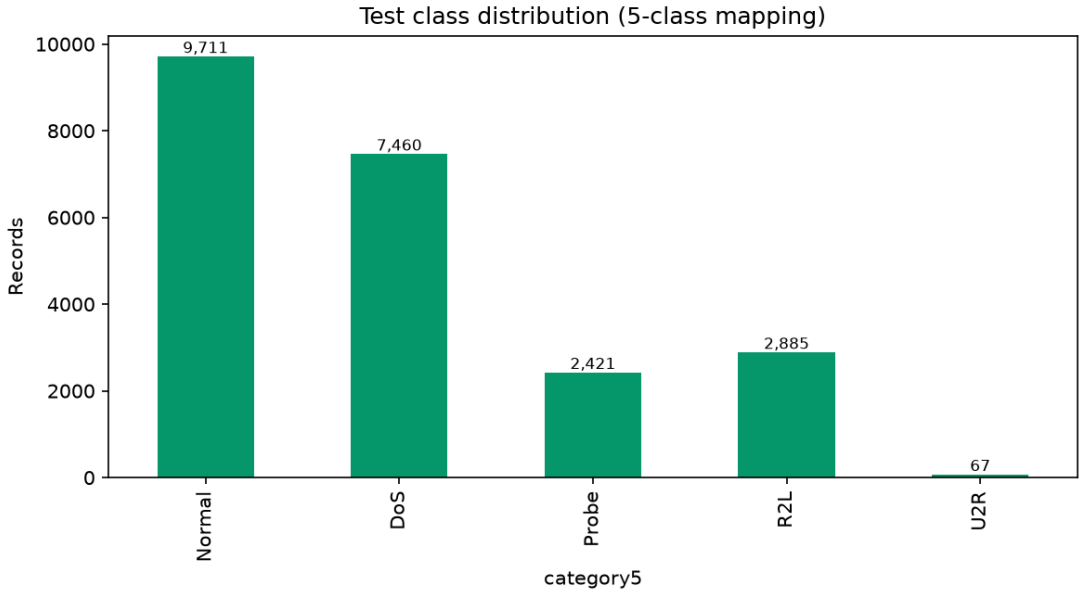


Figure 2 — Test distribution: R2L rises to 12.8% (novel R2L attacks), stressing minority recall.

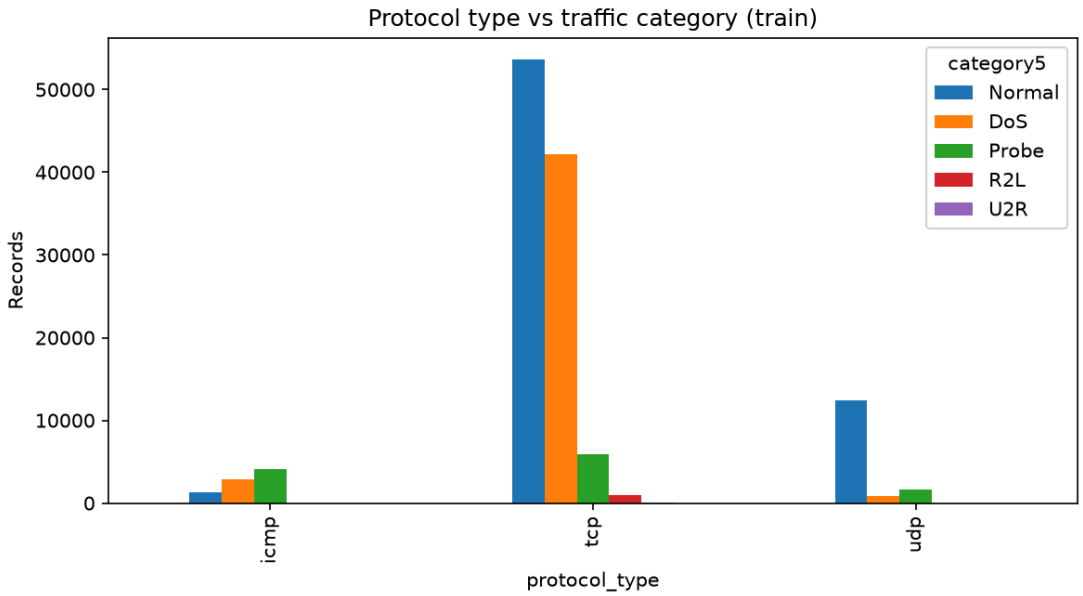


Figure 3 — Protocol vs category: icmp concentrates in DoS/Probe; tcp carries almost all R2L/U2R and most Normal.

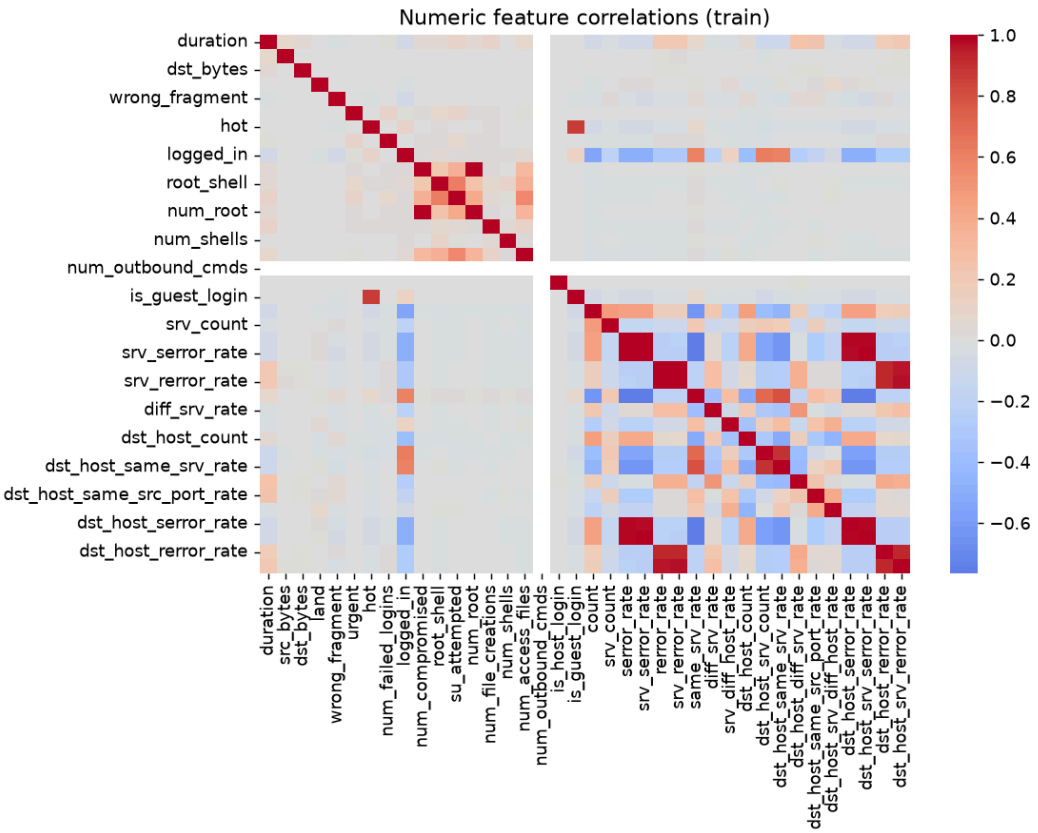


Figure 4 — Numeric correlations: serror/srv_serror and rerror/srv_rerror pairs are strongly correlated; count/srv_count and dst_host aggregates form blocks — tree models tolerate this, linear models less so.

Finding	Evidence	Impact on modelling
Severe imbalance	U2R 52 train rows; R2L 995	Macro metrics essential; class_weight='balanced' used; expect low minority recall
Train/test shift	17 test-only attack types; R2L share 0.8%→12.8%	Test is harder than CV; CV≈0.86 macro-F1 but test ≈0.66
Rare-class signal	R2L/U2R almost all tcp; login/file/root features sparse but decisive	Keep sparse features; don't drop for variance

Correlated rates	serror family block correlation	Prefer ensembles over plain logreg; importance spread

7. Baseline model

Baselines (same train→test protocol): majority-class dummy (trivial reference) and Decision Tree + Logistic Regression (simple ML). Preprocessing identical except scaler for logreg.

Model	Accuracy	Macro-F1	Weighted-F1	Notes
Majority (predict Normal)	0.431	0.120	0.259	Trivial floor; any useful model must clearly beat it
Decision Tree	0.764	0.564	0.726	Interpretable baseline; R2L recall only 0.075
Logistic Regression	0.761	0.569	0.716	Needs scaling; Probe strong (F1 0.82) but R2L collapses (F1 0.03)

8. Model improvement

Compared Random Forest (tuned, class_weight=balanced_subsample) and HistGradientBoosting (class_weight=balanced) with identical features and stratified CV on a 45k train subsample, then re-fit winners on full train and scored once on held-out test. Randomized search over n_estimators {200,350}, max_depth {None,25,40},

min_samples_leaf {1,2} (4 draws, 2-fold, macro-F1). All experiments logged including failures (plain RF overfit CV 0.83→test 0.52 on minorities; logreg R2L failure).

Model	CV acc (3-fold)	CV macro-F1	Test macro-F1	Test acc
Random Forest	0.998	0.827 ± 0.028	0.524	0.745
HistGradient Boosting	0.997	0.860 ± 0.039	0.665	0.801

Best RF params: n_estimators=200, max_depth=40, min_samples_leaf=2 (CV macro-F1 0.883 on search split). CV≫test gap confirms anticipated novelty penalty, not leakage (transforms fit per-fold). Final selection: HistGradientBoosting — best test macro-F1 (0.665 vs 0.524), best accuracy (0.801), better Probe/R2L/U2R recall, sub-millisecond latency, single CPU-friendly estimator. Saved as models/nids_pipeline.joblib.

9. Final model

Pipeline: ColumnTransformer [OneHotEncoder(handle_unknown='ignore') on protocol_type/service/flag + passthrough numerics] → HistGradientBoostingClassifier(class_weight='balanced', random_state=42). Mean inference latency 0.02 ms/record (500-record batch), well within the ~1 s single-record target.

10. Evaluation (held-out KDDTest+, n=22,544)

Metric	Result	What it means here
Accuracy	0.801	4 in 5 records correct overall
Macro precision / recall / F1	0.821 / 0.623 / 0.665	Treats all 5 classes equally; recall short of the 0.80 proposal target — driven by rare classes
Weighted F1	0.784	Frequency-weighted view; closer to user experience on typical traffic
Normal precision	0.714	Below the 0.90 target: ~29% of predicted-Normal traffic needs care — false negatives concentrate in R2L
Attack recall	DoS 0.788, Probe 0.792, R2L 0.282, U2R 0.284	DoS/Probe strong; R2L/U2R miss most novel variants — the honest headline limitation

Class	Precision	Recall	F1	Support
Normal	0.714	0.971	0.823	9,711
DoS	0.963	0.788	0.866	7,460
Probe	0.812	0.792	0.802	2,421
R2L	0.960	0.282	0.437	2,885
U2R	0.655	0.284	0.396	67

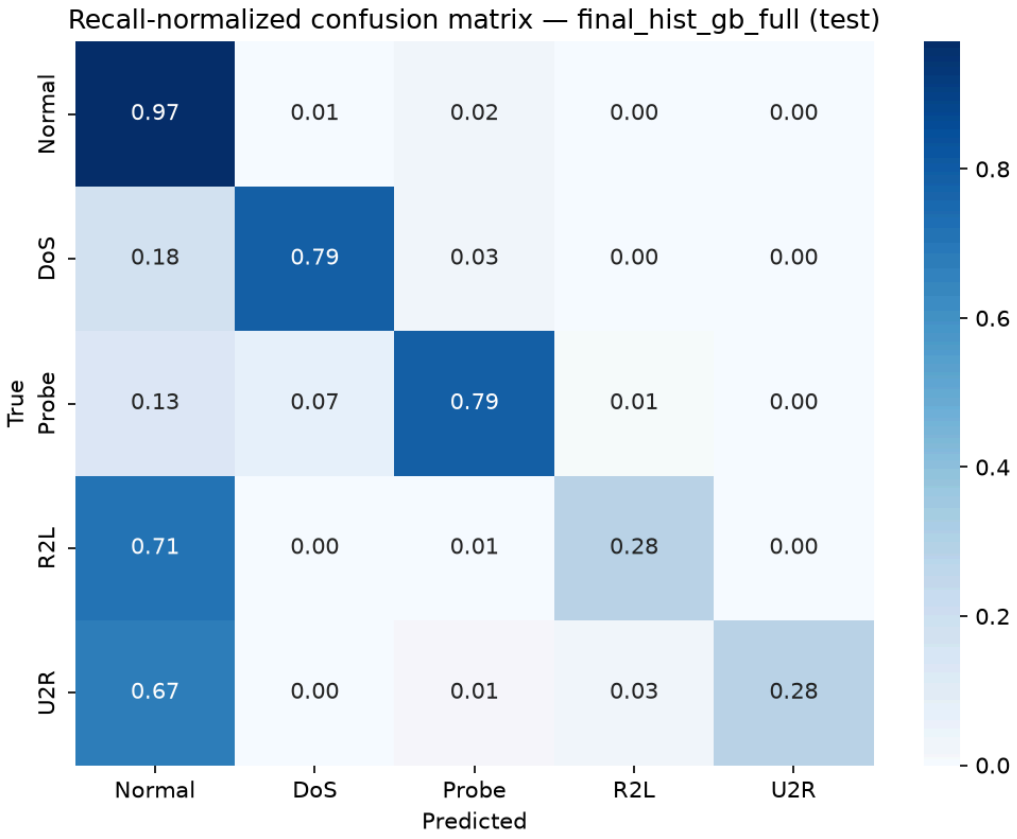


Figure 5 — Recall-normalized confusion matrix (final HGB, test): Normal/DoS/Probe strongly diagonal; R2L row scatters toward Normal; U2R row splits.

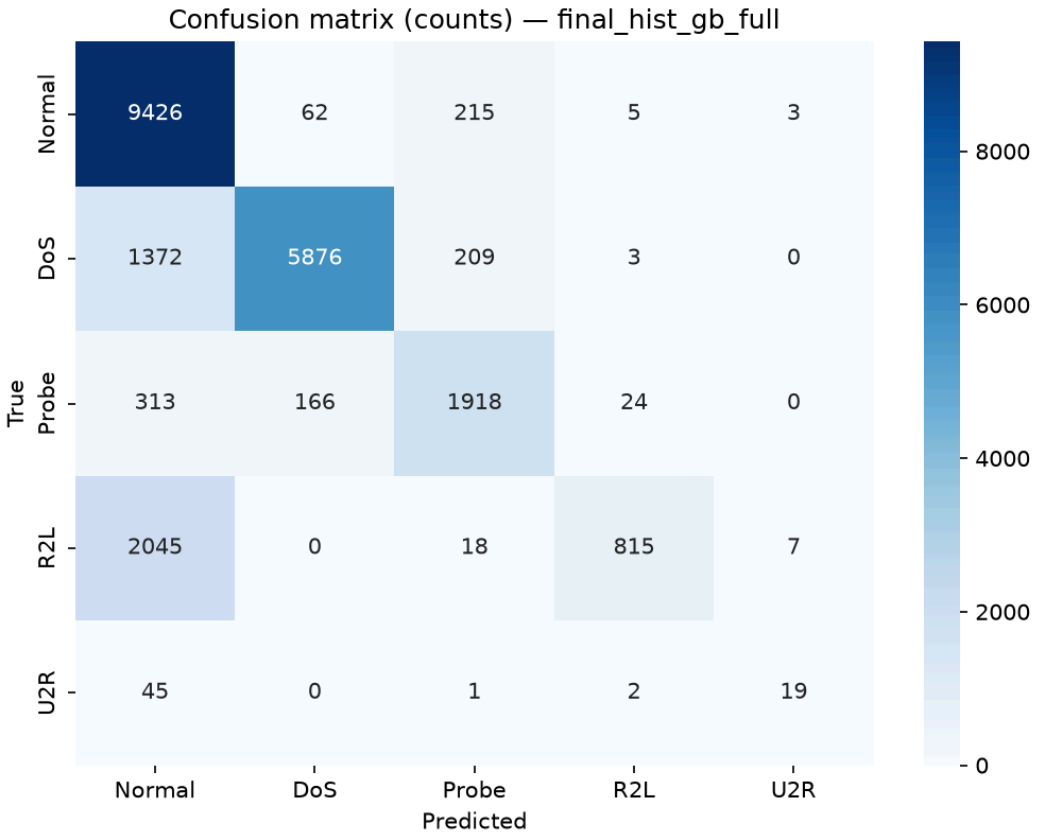


Figure 6 — Confusion counts: 2,070 of 2,885 R2L predicted Normal — the dominant, dangerous error mode.

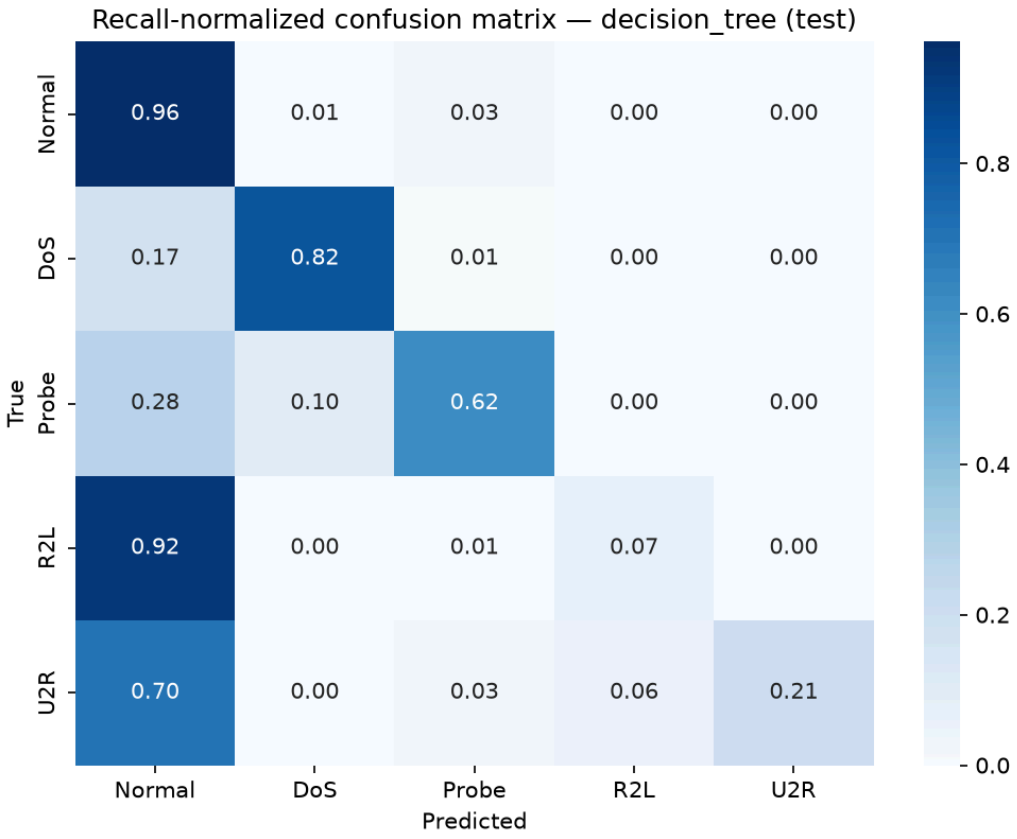


Figure 7 — Baseline Decision Tree recall matrix for comparison (R2L recall 0.07 vs 0.28 final).

Interpretation: an F1 of 0.665 shows moderate multi-class balance on this novelty-heavy test sample, within the limits of a 2009 benchmark — not '66% perfect'. Per-class view matters more than the single number: the prototype reliably flags flood/scan behaviour but cannot be trusted to clear R2L-style login attacks; every Normal prediction on login/file activity needs analyst review. Proposal targets (macro-F1 ≥ 0.80, macro recall ≥ 0.80, Normal precision ≥ 0.90) were not met on this test set; the gap is documented, explained by novel R2L/U2R variants and <1% train representation, and carried into the app disclaimer and §13 limitations rather than tuned away by peeking at test.

11. Testing (whole system)

test_system.py exercises input → validation → preprocessing → model → output (6/6 passed):

Test case	Input	Expected	Actual	Pass?
Normal complete	Sample Normal HTTP record	Prediction, no crash	Normal, p≈0.997	Pass
Missing value	service=None	Handled / message	Handled → Normal	Pass
Invalid number	src_bytes='not_a_number'	Rejected safely	ValueError, safe message	Pass
Unexpected category	service='never_seen_xyz'	Handled	Ignored as unknown → Normal	Pass
Extreme value	src_bytes=999999999	Handled/warned	Prediction returned	Pass
Batch CSV	3-row sample file	3 predictions	Normal, DoS, Normal	Pass

App-level checks: template download, missing-column error, batch bar chart and prediction download verified manually. Note case 2/4 fall back gracefully because the encoder ignores unknowns the app still warns that confidence is not a guarantee.

12. Working prototype

app/app.py (Streamlit): Tab 1 single-record form (all 41 features, sensible defaults) → Classify returns predicted category, per-class probability chart/table, latency and analyst-review warning. Tab 2 batch CSV (template download, missing-column guard, predictions + probabilities table, distribution chart, download). Run: pip install -r requirements.txt; streamlit run app/app.py (from workspace root so models/ resolves).

Demo rows in app/sample_inputs.csv cover Normal / DoS(S0 sweep) / R2L-flavoured FTP. No blocking, no packet capture, no external calls demonstration/pilot only.

13. Limitations and responsible-use notes

1) Historical benchmark no encrypted, cloud, IoT or 2026 attack coverage; do not claim live-network performance. 2) Minority recall gap (R2L 0.28, U2R 0.28) missed intrusions are the top harm; use as triage support with human review, never auto-block. 3) Normal precision 0.71 false assurance risk; surface probabilities and per-class test cards in the UI. 4) Correlated/sparse features and train/test shift mean retraining on local labelled traffic is required before any pilot. 5) Prototype has no auth, audit log, or hardening restrict access, use only benchmark data. 6) Binary ARFF provided file was not used for training (label granularity) documented under Rule #1.

14. Future improvements

Improvement	Limitation addressed	Why postponed
Modern dataset (CIC-IDS2017/2018, UNSW-NB15)	Stale attack coverage	Extra prep/validation time
Streaming ingestion + drift monitoring	Static-record MVP; patterns change	Packet infra + long-term data needed
SIEM/firewall integration + alert ranking	Standalone prototype	APIs, auth, operational testing
Analyst feedback loop + retraining	Fixed model	Feedback pipeline + labels needed
Explainable-AI (SHAP) per prediction	Global importance only	Implementation + interpretation care
Auth, audit logging, hardening	Open demo	Security review scope

15. Conclusion

Module 5 delivers a systematic, reproducible implementation of the approved proposal: documented data and mapping, EDA with interpretations, trivial + simple baselines, justified HGB final model (test acc 0.80, macro-F1 0.66, weighted-F1 0.78, ~0.02 ms/record), honest error analysis centred on novel R2L/U2R, whole-system tests (6/6), and a working Streamlit prototype with safe input handling and clear decision-support framing. The headline caveat strong DoS/Probe triage but unreliable R2L/U2R clearance on novel variants is stated in the model card, the app, and this report.

16. References

1. Tavallaee, M., Bagherzadeh, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. Proc. IEEE Symp. CISDA. 2. Canadian Institute for Cybersecurity, UNB — intrusion detection datasets. 3. NIST SP 800-61 Computer Security Incident Handling Guide. 4. Scikit-learn documentation: preprocessing, ensembles, model evaluation. 5. NSL-KDD benchmark (Kaggle / UNB / defcom17 mirror — cite the mirror actually used); provided course file KDDTest_binary.arff (binary-collapsed variant, inspection only).